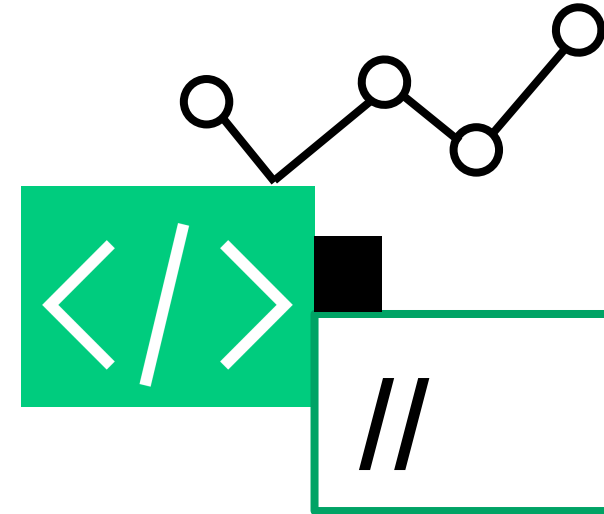


Introducción a TensorFlow y PyTorch

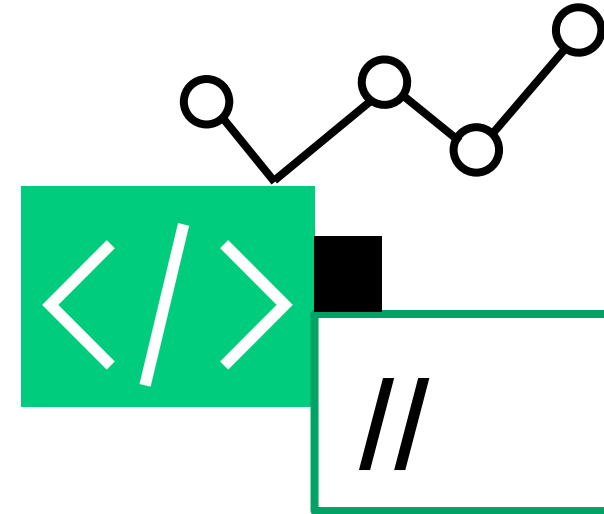
04

Introducción a PyTorch



Entrenar un modelo

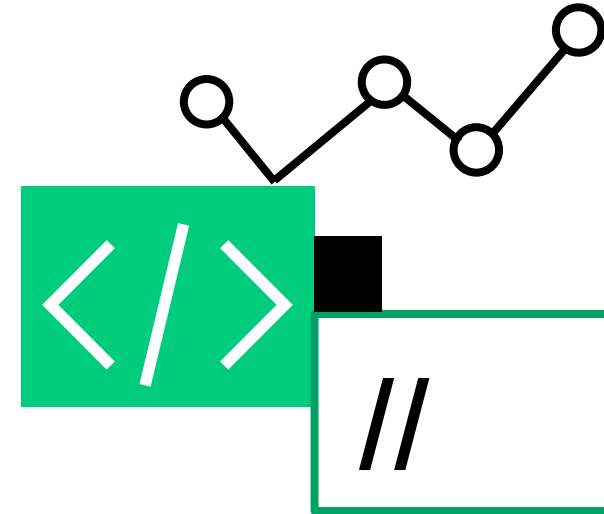
```
model = MyNeuralNetwork()
for epoch in range(2):
    for input, target in trainloader:
        optimizer.zero_grad()
        outputs = model(input)
        loss = criterion(outputs, target)
        loss.backward()
        optimizer.step()
```



Entrenar un modelo

```
model = MyNeuralNetwork()
for epoch in range(2):
    for input, target in trainloader:
        optimizer.zero_grad()
        outputs = model(input)
        loss = criterion(outputs, target)
        loss.backward()
        optimizer.step()
```

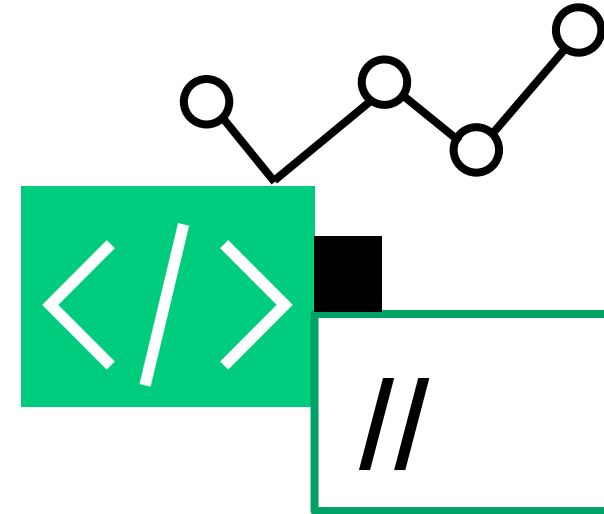
Crea una instancia del modelo



Entrenar un modelo

```
model = MyNeuralNetwork()
for epoch in range(2):
    for input, target in trainloader:
        optimizer.zero_grad()
        outputs = model(input)
        loss = criterion(outputs, target)
        loss.backward()
        optimizer.step()
```

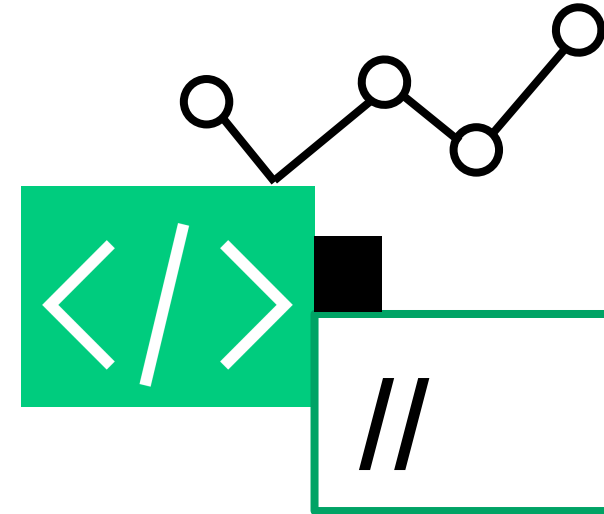
Iterar por un número de épocas de entrenamiento



Entrenar un modelo

```
model = MyNeuralNetwork()
for epoch in range(2):
    for input, target in trainloader:
        optimizer.zero_grad()
        outputs = model(input)
        loss = criterion(outputs, target)
        loss.backward()
        optimizer.step()
```

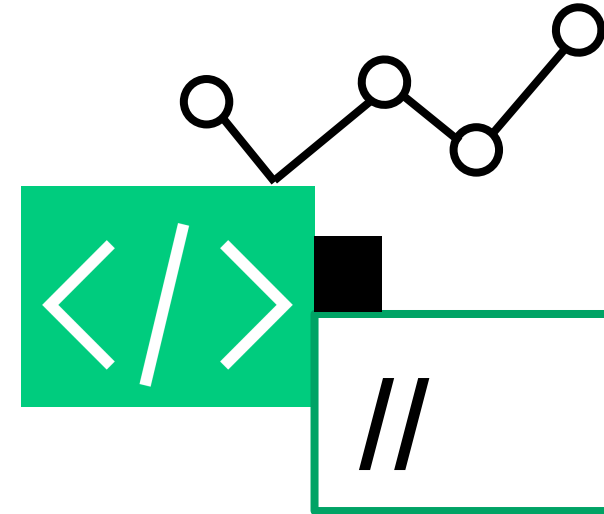
Se realiza para cada batch del conjunto de datos de entrenamiento



Entrenar un modelo

```
model = MyNeuralNetwork()
for epoch in range(2):
    for input, target in trainloader:
        optimizer.zero_grad()
        outputs = model(input)
        loss = criterion(outputs, target)
        loss.backward()
        optimizer.step()
```

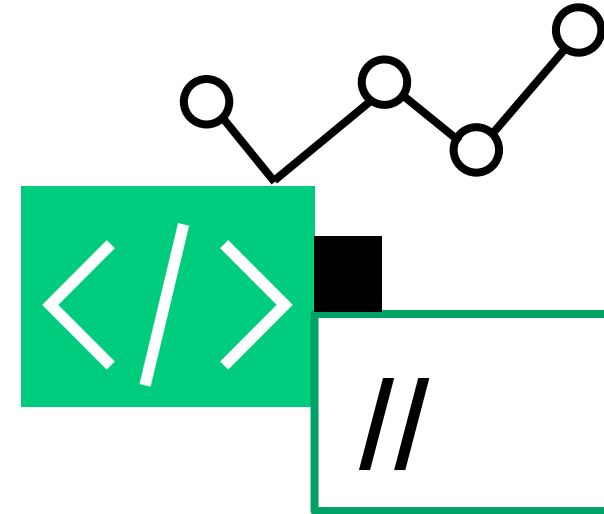
Resetea los gradientes de todos los parámetros del modelo



Entrenar un modelo

```
model = MyNeuralNetwork()
for epoch in range(2):
    for input, target in trainloader:
        optimizer.zero_grad()
        outputs = model(input)
        loss = criterion(outputs, target)
        loss.backward()
        optimizer.step()
```

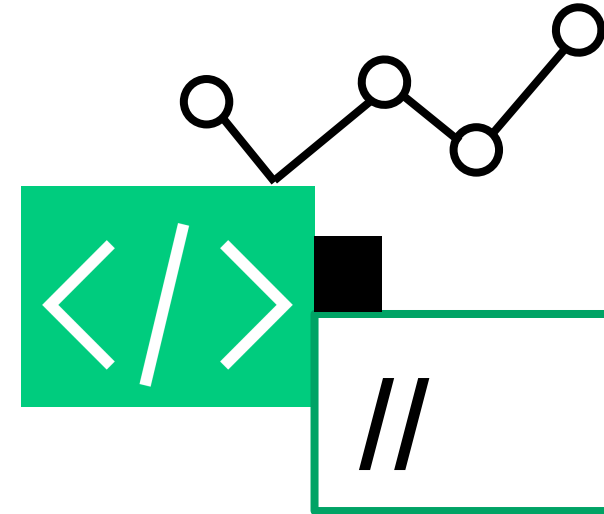
Se calcula la salida dada una entrada de datos de la red neuronal



Entrenar un modelo

```
model = MyNeuralNetwork()
for epoch in range(2):
    for input, target in trainloader:
        optimizer.zero_grad()
        outputs = model(input)
        loss = criterion(outputs, target)
        loss.backward()
        optimizer.step()
```

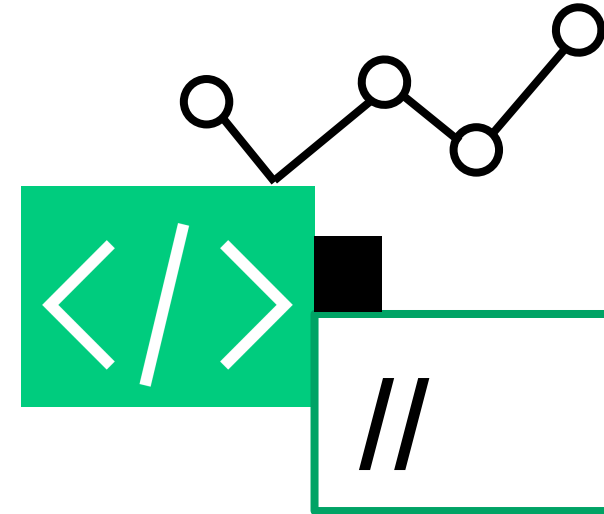
Se calcula el error (la pérdida)



Entrenar un modelo

```
model = MyNeuralNetwork()
for epoch in range(2):
    for input, target in trainloader:
        optimizer.zero_grad()
        outputs = model(input)
        loss = criterion(outputs, target)
        loss.backward()
        optimizer.step()
```

Se realiza la retropropagación



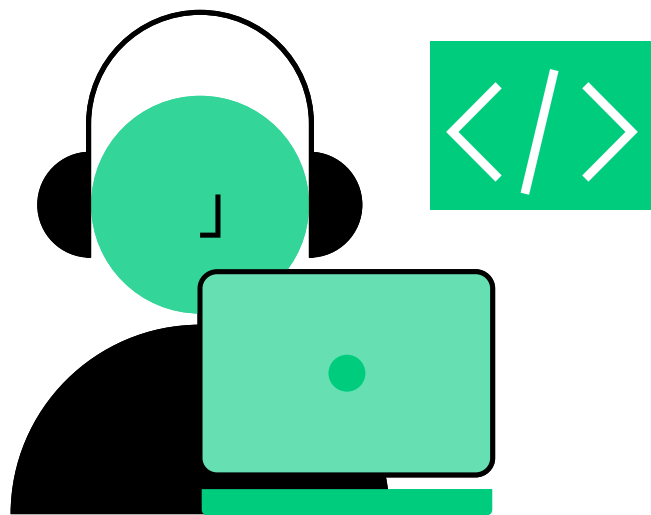
Entrenar un modelo

```
model = MyNeuralNetwork()
for epoch in range(2):
    for input, target in trainloader:
        optimizer.zero_grad()
        outputs = model(input)
        loss = criterion(outputs, target)
        loss.backward()
        optimizer.step()
```

Actualiza los parámetros del modelo

Evaluar un modelo

Requiere determinar un conjunto de datos de validación. **Datos no utilizados en la etapa de entrenamiento**, para evitar sesgos. El tiempo requerido para la evaluación es proporcional al tamaño del conjunto de datos de validación.



```
model.eval()
correct = 0
total = 0

with torch.no_grad():
    for data, target in test_loader:
        outputs = model(data)
        _, predicted = torch.max(outputs.data, 1)
        total += target.size(0)
        correct += (predicted == target).sum().item()

accuracy = 100 * correct / total
print(f'Test Accuracy: {accuracy:.2f}%')
```

¡Muchas gracias!